

Diffusion-TS Interpretable Diffusion for General Time Series Generation

会议：ICLR 2024

作者：Xinyu Yuan, Yan Qiao

单位：Hefei University of Technology

来源：<https://github.com/Y-debug-sys/Diffusion-TS>

一、研究背景

1.1 研究价值

时间序列生成广泛应用于金融、医疗等领域，是解决真实数据稀缺、隐私受限及高成本问题的关键。构建兼具真实性、多样性、可解释性与任务灵活性的高质量模型，对于突破数据获取瓶颈具有重要研究价值。

1.2 现有方法的根本局限与核心矛盾

在研究本文之前，一个根本性问题在于：为何强大的扩散模型在图像生成上效果卓越，在时间序列上却差强人意？存在以下三个“根本矛盾”：

- "黑箱"去噪 vs. "白箱"结构**：时间序列具有明确的趋势、周期性、残差内在结构。标准的U-Net去噪器作为黑箱，在去噪时无差别地恢复所有特征，缺乏对时序可加性分解这一数学结构的“意识”，导致生成序列的结构保真度低。
- "通用"主干 vs. "时序"特性**：许多研究简单移植图像扩散的U-Net或RNN。RNN的自回归导致误差累积与推理缓慢；普通Transformer则缺少对时序周期性的归纳偏置。模型未被“告知”时间序列存在重复模式这一核心特性。
- "特定"训练 vs. "通用"需求**：如CSDI等模型为特定任务（如插补）设计，若要应用于预测或数据增强则需重新训练，违背了我们对“基础生成模型”的期待——即通过不同“提示”灵活适应下游任务。

因此，Diffusion-TS 的核心命题不是“如何适配”，而是“如何重构”：即重新设计扩散模型，使其内在机制与时间序列的本质属性（可分解性、周期性、长程依赖）对齐。

二、研究目的

2.1 研究目标

- 构建一个适用于多变量、长时间序列的扩散生成模型；
- 在生成过程中显式建模趋势与季节性结构，实现可解释生成；
- 在不重新训练模型的情况下，统一支持无条件生成与条件任务（预测、补全）。

作者试图证明：

通过将经典时间序列分解思想引入扩散模型，可以在不牺牲生成质量的前提下，显著提升模型

- 处理后的特征再与解码器的中间输出向量进行逐元素相加，以整合信息。

对比思考：

- vs RNN：Transformer 解决了 RNN 的误差累积和无法并行问题。
- vs Standard U-Net：U-Net 是局部卷积为主，Transformer 的 Self-Attention 能捕捉序列首尾的长程依赖（比如一年的年初和年尾的趋势关系）。

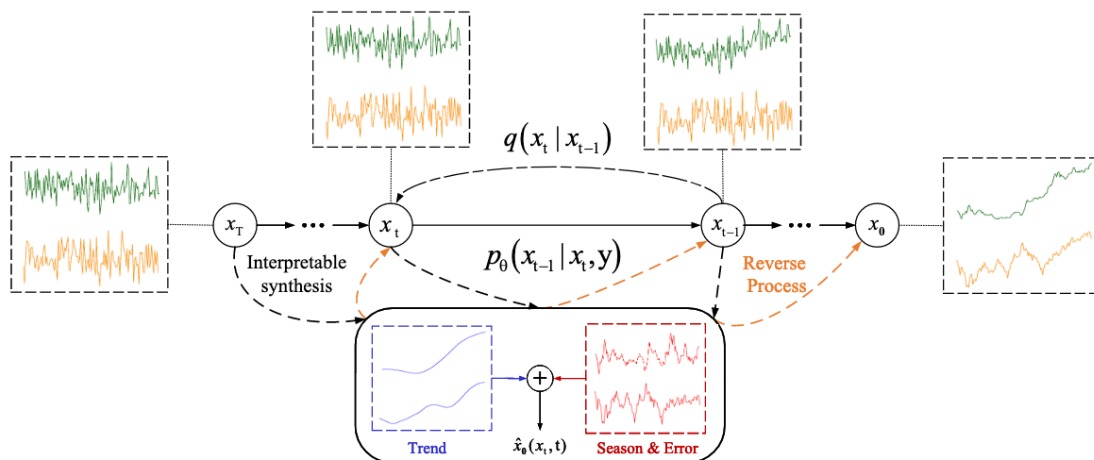


Figure 1: The forward and reverse diffusion on time series data in Diffusion-TS. The denoising network learns to predict the clean time series $\hat{x}_0$ from x_t based on interpretable decomposition. In the reverse pass, the generator network gradually injects the expert knowledge to make the synthetic series target move toward the real ones.

3.2 关键组件一：趋势合成层 (Trend Synthesis)

设计动因：从含噪数据中抽取出平滑的、缓慢变化的基线。

操作方法

不使用神经网络直接拟合数值，而是用多项式回归。为了强制“平滑”，作者引入了一个强归纳偏置：低阶多项式回归。不直接预测数值，而是预测多项式的系数，利用系数画出低频的平滑曲线。

公式详解

$$V_{tr}^t = \sum_{i=1}^D (C \cdot \text{Linear}(w_{tr}^{i,t}) + \mathcal{X}_{tr}^{i,t}), \quad C = [1, c, \dots, c^p]$$

- 多项式基矩阵：** $C = [1, c, c^2, \dots, c^p]$
 - c 是一个归一化的时间索引向量（如从-1到1）
 - p 是多项式阶数（如3），用于模拟低频行为
 - 这个矩阵定义了所有可能的、最高 p 阶的平滑曲线“模板”
- 权重学习：** $\text{Linear}(w_{tr}^{i,t})$
 - 这是第 i 个解码器块的中间特征，经过一个线性层后的输出
 - 可以理解为每个时间点、每个通道上学到的 $p+1$ 个权重系数

3. 多项式回归核心: $C \cdot \text{Linear}(w_{tr}^{i,t})$

- 对于每个时间点, 用学到的权重去线性组合多项式基 $[1, c, c^2, \dots]$
- 从而生成一条唯一确定的、平滑的曲线
- 矩阵乘法实现了对所有时间点的并行计算

4. 偏置项: $+x_{tr}^{i,t}$

- 这是一个偏置项, 通常取解码器块输入的均值
- 确保趋势线有一个合理的基准位置

5. 累加融合: $\sum_{i=1}^D$

- 将所有解码器层产生的趋势分量累加起来
- 不同层可能捕捉不同时间尺度或抽象层次的趋势, 累加使得趋势表达更丰富

思考与必要性

1. **必要性:** 趋势本质是低频的。如果用全连接层拟合, 会把高频噪声也当成趋势学进去。
2. **对比:** 相比 TimeGAN 使用 RNN 拟合趋势, 这里限制 $p = 3$ 相当于加了一个硬约束, 强制输出必须是平滑曲线。
3. **缺陷:** 如果真实数据的趋势包含剧烈突变 (如股市熔断), 低阶多项式无法拟合。
4. 代码实现:

```
for i, dec_layer in enumerate(self.decoder):
    dec_out = ... # 解码器块的输出
    trend_feat = self.to_trend(dec_out) # [batch, seq_len, trend_out_dim]# 线性变换 Linear(w_tr)
    trend_component = torch.einsum('t p, b t d -> b t d', self.poly_basis, trend_feat) # 与多项式基矩阵 C 相乘 (实现 C · Linear(w))
    trend_component = trend_component + dec_out.mean(dim=1, keepdim=True) # 加上偏置项 (第 i 块输出的均值)
    total_trend += trend_component # 累加到总趋势中
```

3.3 关键组件二: 季节性合成层 (Seasonality Synthesis)

设计目标: 自适应频率选择器

任务是**从去除趋势后的信号中, 找出并重建重复出现的周期性模式。**

操作方法

残差 -> FFT -> Top-K 滤波 -> iFFT

公式详解

$$A_{i,t}^{(k)} = |\mathcal{F}(w_{seas}^{i,t})_k|, \quad \Phi_{i,t}^{(k)} = \phi(\mathcal{F}(w_{seas}^{i,t})_k)$$
$$\kappa_{i,t}^{(1)}, \dots, \kappa_{i,t}^{(K)} = \arg \text{TopK}_{k \in \{1, \dots, \lfloor \tau/2 \rfloor + 1\}} \{A_{i,t}^{(k)}\}$$
$$S_{i,t} = \sum_{k=1}^K A_{i,t}^{\kappa_{i,t}^{(k)}} [\cos(2\pi f_{\kappa_{i,t}^{(k)}} \tau c + \Phi_{i,t}^{\kappa_{i,t}^{(k)}}) + \cos(2\pi \bar{f}_{\kappa_{i,t}^{(k)}} \tau c + \bar{\Phi}_{i,t}^{\kappa_{i,t}^{(k)}})]$$

1. FFT变换: $A, \Phi = \text{FFT}(\text{Input})$

- 计算各频率的振幅和相位

2. Top-K选择: $\text{Indices} = \text{argTopK}(A, K)$

- 只保留能量最大的 K 个频率，认为这些是主要的周期信号，其余为噪声。

3. 重构:

$$S_t = \sum_{k \in \text{Indices}} A_k \cos(2\pi f_k t + \Phi_k)$$

- 用筛选出的 K 个主频率，以其对应的振幅和相位，在时域重构出季节性信号。

思考与必要性

1. **必要性**: 真实数据的周期往往是多重的（日、周、年）且非整数。
2. **对比**: 传统 STL 分解需人工指定周期（如 "period=24"）。本文方法是数据驱动的，能自动发现主导频率。
3. **缺陷**: 傅里叶变换假设信号在窗口内是平稳的。对于变频信号（频率随时间改变），FFT 效果未必好。
4. 代码实现:

```
total_seasonal = 0
for i, dec_layer in enumerate(self.decoder):
    dec_out = ... # 解码器块的输出
    seasonal_feat = self.to_seasonal(dec_out)
    feat_fft = torch.fft.rfft(seasonal_feat, dim=1) # 傅里叶变换 (FFT) 获取振幅和相位
    amplitude = torch.abs(feat_fft) # 振幅 A
    phase = torch.angle(feat_fft) # 相位  $\Phi$ 
    topk_vals, topk_indices = torch.topk(amplitude, self.top_k, dim=1) # Top-K
    # 滤波: 选取振幅最大的 K 个频率分量
    reconstructed_fft = torch.zeros_like(feat_fft, device=feat_fft.device) # 用
    # 选中的频率分量重构季节性信号
    reconstructed_fft.scatter_(1, topk_indices, feat_fft.gather(1,
    topk_indices)) # 仅将 Top-K 频率的分量填回模板中
    seasonal_component = torch.fft.irfft(reconstructed_fft,
    n=seasonal_feat.size(1), dim=1) # 逆傅里叶变换 (iFFT) 回时域
    total_seasonal += seasonal_component
```

3.4 训练目标: 预测 x_0 + 频域损失

• 设计动因:

1. **范式转移**: 传统 DDPM 预测噪声 ϵ ，其目标与最终期望的干净数据 x_0 是间接的。既然网络已明确输出 x_0 的分解成分，**直接预测 x_0 是更自然、更“端到端”的选择。**
2. 仅凭时域 MSE 损失可能无法充分捕捉周期性特征（如相位信息）。

总结以上两点:

- 原始扩散模型的反向过程目标是从噪声 x_T 中通过去噪过程还原出原始数据 x_0 。这个过程本质上是无条件的，模型的核心是学习原始数据分布 $p(x_0)$ 。
- Diffusion-TS的一个关键设计是修改了训练目标，它不直接预测噪声 ϵ ，而是训练神经网络直接预测一个对原始样本的估计，并结合了一个傅里叶基损失项来增强频域重建的准确性。这个设计迫使模型同时优化时域和频域特征。这一设计的关键优势在于，解决了单一时域损失MSE在捕获复杂周期模式上的不足。MSE项保证生成序列在时间维度上的基本保真度，而FFT损失则专门确保其周期性行为的真实性。

• 具体方法与公式：

1. 预测 $\hat{x}_0$ ：网络输出直接是干净样本的分解合成结果。最终，原始信号的估计 $\hat{x}_0$ 由趋势、季节性和残差（由最后一个解码器块的输出 R 表示）相加重构而成，这种显式的、基于公式的分解是模型可解释性和高生成质量的关键来源。

2. 时频联合优化损失：

$$\mathcal{L}_\theta = \mathbb{E}_{t, x_0} [w_t [\lambda_1 \|x_0 - \hat{x}_0(x_t, t, \theta)\|^2 + \lambda_2 \|\mathcal{FFT}(x_0) - \mathcal{FFT}(\hat{x}_0(x_t, t, \theta))\|^2]]$$

其中， w_t 是扩散步 t 的权重， λ_1, λ_2 是平衡超参数。

$$Weight_t = \frac{\sqrt{\alpha_t} \cdot \sqrt{1 - \bar{\alpha}_t}}{100 \cdot \beta_t}$$

损失函数包含两部分：

3. 时域损失 (第一项)：标准的均方误差 (MSE)，保证数值上的准确性。
4. 频域损失 (第二项)：对预测结果和真实结果做 FFT 后计算误差。这强制模型生成的序列在频率分布（周期性）上与真实数据一致，防止相位漂移或频率丢失。

引入权重：不同时间步的重要性不同，进行加权，防止模型过度关注某些容易学的阶段或被高噪声大误差主导。

• 代码实现：

```
def train_step(self, x0):
    ...
    # 扩散过程：随机选择时间步 t，并加噪得到 x_t
    t = torch.randint(0, self.num_timesteps, (x0.shape[0],),
device=x0.device).long()
    x_t = self.q_sample(x0, t) # 前向加噪

    # 模型重建：预测 x_0（对应上述设计动因1）
    x0_recon = self.model(x_t, t)

    # 1. 时域 MSE 损失（对应损失公式中的第一项）
    mse_loss = F.mse_loss(x0_recon, x0, reduction='mean')

    # 2. 频域傅里叶损失（对应损失公式中的第二项）
    fft_real = torch.fft.rfft(x0, dim=1)
    fft_recon = torch.fft.rfft(x0_recon, dim=1)
    freq_loss = F.mse_loss(torch.abs(fft_recon), torch.abs(fft_real))

    # 3. 加权总损失
    total_loss = mse_loss + self.lambda_freq * freq_loss # lambda_freq
```

对应 λ_2

...

• 优点:

1. **目标-架构对齐**: 训练目标 (预测 x_0x_0) 与网络架构的输出 (合成 x_0x_0) 完全一致, 体现了协同设计思想。
2. **频域双重保障**: 傅里叶损失作为强正则项, 在频域约束生成的季节性成分, 确保其**频率构成与能量分布与真实数据一致**, 有效防止相位漂移和频率丢失, 是对傅里叶合成层的有效补充。

3.5 即插即用的条件生成

设计动因

为了实现"一个预训练模型, 支持多种下游任务 (无条件生成、条件生成、预测、插补)"的愿望, 必须有一种在采样阶段融入条件信息, 而无需重新训练模型的方法。

方法实现: 重构引导采样

为了支持条件生成, Diffusion-TS引入了分类器引导机制。该方法通过后验概率分布

$p(x_{0:T}|y) = \prod_{t=1}^T p(x_{t-1}|x_t, y)$ 进行近似采样, 并基于

$p(x_{t-1}|x_t, y) \propto p(x_{t-1}|x_t) \cdot p(y|x_{t-1}, x_t)$ 推导梯度计算过程。相应的对数概率梯度分解为:

$$\nabla_{x_{t-1}} \log p(x_{t-1}|x_t, y) = \nabla_{x_{t-1}} \log p(x_{t-1}|x_t) + \nabla_{x_{t-1}} \log p(y|x_{t-1}, x_t)$$

其核心改进是在原始扩散模型的反向传播中加入了外部条件 y , 引导生成过程产生符合条件的时间序列。其中, 无条件扩散模型的先验概率确保生成数据符合真实分布, 而分类器梯度则评估当前中间结果 x_{t-1} 符合目标条件 y 的程度, 并提供调整方向。梯度信号由预训练的判别模型提供。

虽然借鉴了分类器引导, 但区别在于用模型自身的重建误差代替了需要额外训练的分类器。这种设计具有灵活性: 同一个预训练的Diffusion-TS模型只需在采样时更换引导函数 (如从插值条件改为预测条件), 即可适应不同下游任务, 实现"即插即用"。

以插补任务 (已知部分时间点 x_a , 生成其余部分) 为例, 在采样的每一步 t :

梯度引导公式详解

$$\tilde{x}_0 = \hat{x}_0 + \eta \cdot \nabla_{x_t} \left(\underbrace{\|M \odot (y - \hat{x}_0)\|^2}_{\text{条件一致性}} + \underbrace{\gamma \log p(x_t)}_{\text{真实性先验}} \right)$$

解释:

1. η (Step size): 引导力度。太大生成的会崩坏, 太小条件不管用。
2. ∇_{x_t} : 求导。意思是"为了减小当前生成结果与条件 y 的误差, 输入噪声 x_t 应该怎么改?"
3. M (Mask): 掩码, 指示哪些部分是已知条件。


```
def `p_sample_guid`ed(self, x_t, t, x_known, known_indices,
    guidance_scale=1.0):`
```

```
with torch.enable_grad():
    x_t_in = x_t.detach().requires_grad_(True)
    # 1. 无条件估计
    x0_pred = self.model(x_t_in, t)
    # 2. 计算已知部分的重建误差作为引导信号（对应公式12第一项）
    known_pred = x0_pred[:, known_indices, :]
    loss_guidance = F.mse_loss(known_pred, x_known, reduction='sum')
    # 3. 梯度引导更新
    grad = torch.autograd.grad(loss_guidance, x_t_in)[0]
    x_t_modified = x_t_in + guidance_scale * grad # guidance_scale 对应  $\eta$ 
    sqrt_alpha_bar_t = self.sqrt_alphas_cumprod[t] # 硬约束替换
    sqrt_one_minus_alpha_bar_t = self.sqrt_one_minus_alphas_cumprod[t]
    noise = torch.randn_like(x_known)
    # 实现  $x_a^{\{(t-1)\}}$  的计算
    x_t_known_correct = sqrt_alpha_bar_t * x_known +
    sqrt_one_minus_alpha_bar_t * noise
    # 执行替换操作
    x_t_modified[:, known_indices, :] = x_t_known_correct
```

```
    # 5. 用修改后的  $x_t$  进行标准的反向扩散一步
    x_prev = self.p_sample(x_t_modified, t)
```

```
return x_prev
```

```
def conditional_sample_loop(self, shape, x_known, known_indices):
    ...
    for t in reversed(range(self.num_timesteps)):
        # 动态确定当前步的梯度更新次数  $K[t]$ （对应附录F算法2优化）
        if t > self.num_timesteps * 0.7:
            K_t = 5 # 早期强力引导
        elif t > self.num_timesteps * 0.3:
            K_t = 3
        else:
            K_t = 1 # 后期微调加速
        for _ in range(K_t):
            x_t = self.p_sample_guided(x_t, t, x_known, known_indices, guidance_scale)
    return x_t
```

动态采样算法与优化

扩散模型的反向过程可划分为早期“创造阶段”和后期“平滑阶段”。论文中提出了重构引导采样算法，如算法1所示。该算法从完全噪声 $X_T \sim \mathcal{N}(0, I)$ 开始迭代去噪。在每个时间步，它从原始预测 $\hat{x}_0$ 中分割出对应原始条件数据 x_a 的部分和需生成的部分 x_b ，将两者拼接成完整序列后计算重构损失 L_1 。接着执行一次无条件去噪，使用 $\hat{x}_0$ 和 x_t 计算均值 μ ，并从构造的分布中采样出 x_{t-1} 。随后计算模型先验损失 L_2 （即采样得到的与理论均值的差异）。关键的第7步对应

上述梯度引导更新公式，第8步则执行引导后的去噪过程，重新计算均值并采样得到新的 x_{t-1} 。 $Replace(x_a, x_{t-1}, t)$ 操作将新采样的 x_{t-1} 中对应已知条件 x_a 的部分，用真实条件数据加上时间步 $t-1$ 的对应噪声替换，以确保条件一致性。

Algorithm 1 Reconstruction-guided Sampling	Algorithm 2 Optimized Conditional Sampling
Require: gradient scale η , conditional data x_a , trade-off coefficient γ 1: $x_T \sim \mathcal{N}(0, \mathbf{I})$; 2: for all t from T to 1 do 3: $[\hat{x}_a, \hat{x}_b] \leftarrow \hat{x}_0(x_t, t, \theta)$; 4: $\mathcal{L}_1 = \ x_a - \hat{x}_a\ _2^2$; 5: $x_{t-1} \leftarrow \mathcal{N}(\mu(\hat{x}_0(x_t, t, \theta), x_t), \Sigma)$; 6: $\mathcal{L}_2 = \ x_{t-1} - \mu(\hat{x}_0(x_t, t, \theta), x_t)\ _2^2 / \Sigma$; 7: $\hat{x}_0 = \hat{x}_0(x_t, t, \theta) + \eta \nabla_{x_t}(\mathcal{L}_1 + \gamma \mathcal{L}_2)$; 8: $x_{t-1} \leftarrow \mathcal{N}(\mu(\hat{x}_0, x_t), \Sigma)$; 9: $x_{t-1} \leftarrow Replace(x_a, x_{t-1}, t)$; 10: end for	Require: gradient scale η , conditional data x_a , trade-off coefficient γ , times set K 1: $x_T \sim \mathcal{N}(0, \mathbf{I})$; 2: for all t from T to 1 do 3: for all i from $K[t]$ to 1 do 4: $[\hat{x}_a, \hat{x}_b] \leftarrow \hat{x}_0(x_t, t, \theta)$; 5: $\mathcal{L}_1 = \ x_a - \hat{x}_a\ _2^2$; 6: $x_{t-1} \leftarrow \mathcal{N}(\mu(\hat{x}_0(x_t, t, \theta), x_t), \Sigma)$; 7: $\mathcal{L}_2 = \ x_{t-1} - \mu(\hat{x}_0(x_t, t, \theta), x_t)\ _2^2 / \Sigma$; 8: $x_t = x_t + \eta \nabla_{x_t}(\mathcal{L}_1 + \gamma \mathcal{L}_2)$; 9: end for 10: $x_{t-1} \leftarrow \mathcal{N}(\mu(\hat{x}_0(x_t, t, \theta), x_t), \Sigma)$; 11: $x_{t-1} \leftarrow Replace(x_a, x_{t-1}, t)$; 12: end for

算法2是算法1的优化版本，其核心改进是引入了动态调整的梯度更新次数 $K[t]$ ，该参数表示在扩散时间步 t 重复执行梯度更新的次数。在噪声较多的早期创造阶段，设置较大的 $K[t]$ 值，通过多次梯度更新强条件引导，确保生成样本的基础结构与给定条件高度一致。在噪声较少、数据已较清晰的后期平滑阶段，则设置较小的 $K[t]$ 值，减少不必要的计算开销，加速采样过程，因此阶段过度优化收益已不显著。

操作：前期（噪声大）多做几次梯度更新；后期（噪声小）少做。

- 噪声大 $\rightarrow$ 大 $K[t]$ ，强条件约束
- 噪声小 $\rightarrow$ 小 $K[t]$ ，加速采样

必要性：前期是定骨架阶段，如果骨架歪了（比如趋势预测反了），后期很难救回来。所以前期要强力引导。

四、实验结果

4.1 实验数据集

为了全面评估模型的性能，作者选取了 6 个 涵盖不同领域（金融、能源、医疗、物理模拟）、不同维度和特性的数据集：

- **真实世界数据集：
 - **Stocks**：2004年至2019年的谷歌股票价格数据，包含6个特征，观测粒度为天。
 - **ETTh**：电力变压器数据，包含负载和油温等7个特征，每15分钟记录一次。
 - **Energy**：UCI家电能源预测数据集，包含28个特征。
 - **fMRI**：因果发现基准数据集，包含真实的血氧水平依赖（BOLD）时间序列模拟，具有50个特征（高维数据）。
- 模拟数据集：
 - **Sines**：包含5个特征，每个特征由不同的频率和相位独立生成 6。

- **MuJoCo**: 多变量物理模拟时间序列数据，包含14个特征 7。

4.2 实验效果

实验主要从无条件生成、长序列生成鲁棒性、条件生成（插补与预测）以及可解释性四个方面进行了验证。

(1) 无条件生成表现

- **综合性能 SOTA**: 在标准长度 ($L=24$) 的生成任务中，Diffusion-TS 在所有6个数据集上，针对 Discriminative Score (判别分数)、Predictive Score (预测分数) 等指标均取得了最佳性能。
- **显著提升**: 相比于 TimeGAN、Diffwave 等基线模型，Diffusion-TS 在判别分数上平均提升了约 **50%**，说明生成的样本在分布上更接近真实数据。
- **高维适应性**: 在 MuJoCo、Energy 和 fMRI 等高维数据集上，模型的领先优势更为显著，证明了其处理复杂多变量依赖的能力。

(2) 长序列生成的鲁棒性

- **实验设置**: 在 ETTh 和 Energy 数据集上测试了生成长度为 64、128 和 256 的序列。
- **结果对比**: 随着序列长度增加，传统 RNN 基线（如 TimeGAN）的性能急剧下降，出现了明显的误差累积。
- **模型优势**: Diffusion-TS 的性能保持极度稳定，甚至在长序列 ($L=256$) 上表现更好。这证明了其基于 Transformer 的分解架构能有效捕捉长程依赖，避免了自回归模型的缺陷。

(3) 条件生成表现

- **高缺失率插补**: 在数据缺失率高达 **90%** 的极端测试中，Diffusion-TS 的插补误差显著低于专门设计的 SOTA 插补模型（如 CSDI）。这表明模型凭借强大的结构先验（趋势+季节），在信息极度匮乏时仍能进行合理的全局推理。
- **引导策略有效性**: 对比实验显示，在缺失率较高（75%以上）时，使用 **重构梯度引导 (Reconstruction-Guidance)** 的采样策略比简单的替换策略效果更好。
- **预测任务**: 在 Solar 数据集的长期预测任务中，Diffusion-TS 同样取得了最佳的 MSE 成绩。

(4) 可解释性与冷启动

- **结构解耦**: 可视化结果显示，模型能够准确地将含噪输入分解为平滑的趋势项和周期性的季节项，且分解结果与合成数据的 Ground Truth 高度吻合。
- **数据稀缺鲁棒性**: 在仅使用 **10%** 训练数据的“冷启动”实验中，Diffusion-TS 经过自我修复和不规则训练，性能可恢复至接近全量数据训练的水平。

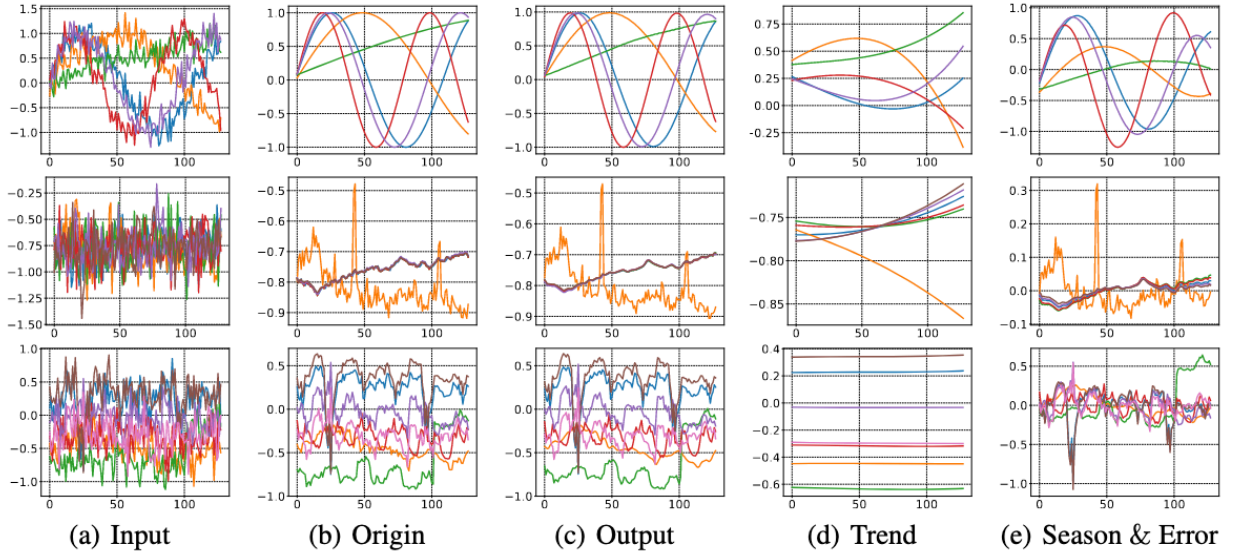


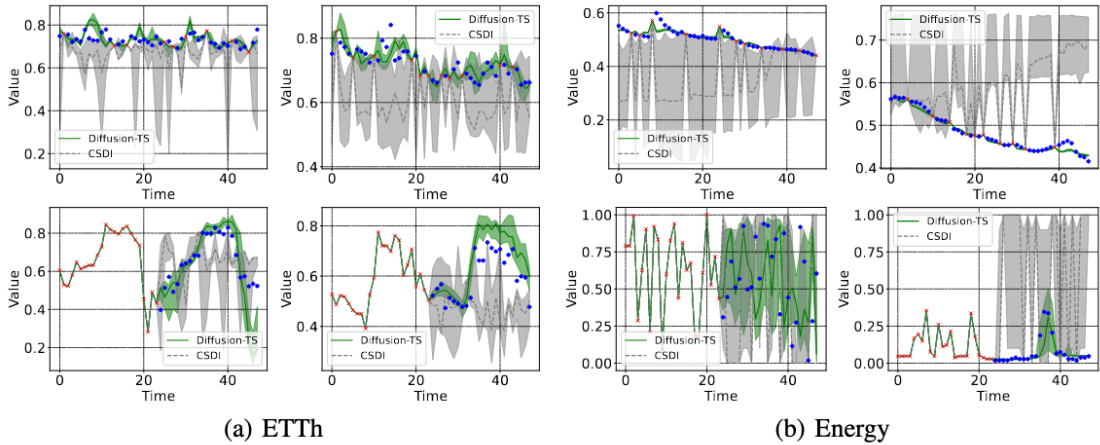
Figure 3: The reconstruction performance of Diffusion-TS with the interpretable configurations. Each row is a time series example per dataset, top to bottom (Sines, Stocks and ETTh).

Table 1: Results on Multiple Time-Series Datasets (Bold indicates best performance).

Metric	Methods	Sines	Stocks	ETTh	MuJoCo	Energy	fMRI
Context-FID Score (Lower the Better)	Diffusion-TS	0.006±.000	0.147±.025	0.116±.010	0.013±.001	0.089±.024	0.105±.006
	TimeGAN	0.101±.014	0.103±.013	0.300±.013	0.563±.052	0.767±.103	1.292±.218
	TimeVAE	0.307±.060	0.215±.035	0.805±.186	0.251±.015	1.631±.142	14.449±.969
	Diffwave	0.014±.002	0.232±.032	0.873±.061	0.393±.041	1.031±.131	0.244±.018
	DiffTime	<u>0.006±.001</u>	0.236±.074	<u>0.299±.044</u>	<u>0.188±.028</u>	<u>0.279±.045</u>	0.340±.015
	Cot-GAN	1.337±.068	0.408±.086	0.980±.071	1.094±.079	1.039±.028	7.813±.550
	Original						
Correlational Score (Lower the Better)	Diffusion-TS	0.015±.004	0.004±.001	0.049±.008	0.193±.027	0.856±.147	1.411±.042
	TimeGAN	0.045±.010	0.063±.005	0.210±.006	0.886±.039	4.010±.104	23.502±.039
	TimeVAE	0.131±.010	0.095±.008	0.111±.020	0.388±.041	1.688±.226	17.296±.526
	Diffwave	0.022±.005	0.030±.020	0.175±.006	0.579±.018	5.001±.154	3.927±.049
	DiffTime	0.017±.004	0.006±.002	0.067±.005	0.218±.031	1.158±.095	1.501±.048
	Cot-GAN	0.049±.010	0.087±.004	0.249±.009	1.042±.007	3.164±.061	26.824±.449
	Original						
Discriminative Score (Lower the Better)	Diffusion-TS	0.006±.007	0.067±.015	0.061±.009	0.008±.002	0.122±.003	0.167±.023
	TimeGAN	0.011±.008	0.102±.021	0.114±.055	0.238±.068	0.236±.012	0.484±.042
	TimeVAE	0.041±.044	0.145±.120	0.209±.058	0.230±.102	0.499±.000	0.476±.044
	Diffwave	0.017±.008	0.232±.061	0.190±.008	0.203±.096	0.493±.004	0.402±.029
	DiffTime	0.013±.006	<u>0.097±.016</u>	<u>0.100±.007</u>	<u>0.154±.045</u>	0.445±.004	<u>0.245±.051</u>
	Cot-GAN	0.254±.137	0.230±.016	0.325±.099	0.426±.022	0.498±.002	0.492±.018
	Original						
Predictive Score (Lower the Better)	Diffusion-TS	0.093±.000	0.036±.000	0.119±.002	0.007±.000	0.250±.000	0.099±.000
	TimeGAN	0.093±.019	0.038±.001	0.124±.001	0.025±.003	0.273±.004	0.126±.002
	TimeVAE	<u>0.093±.000</u>	0.039±.000	0.126±.004	0.012±.002	0.292±.000	0.113±.003
	Diffwave	<u>0.093±.000</u>	0.047±.000	0.130±.001	0.013±.000	0.251±.000	0.101±.000
	DiffTime	<u>0.093±.000</u>	<u>0.038±.001</u>	<u>0.121±.004</u>	<u>0.010±.001</u>	0.252±.000	<u>0.100±.000</u>
	Cot-GAN	0.100±.000	0.047±.001	0.129±.000	0.068±.009	0.259±.000	0.185±.003
	Original						
Original		0.094±.001	0.036±.001	0.121±.005	0.007±.001	0.250±.003	0.090±.001

Table 3: Detailed Results of Long-term Time-series Generation. (Bold indicates best performance).

	Dataset	Length	Diffusion-TS	TimeGAN	TimeVAE	Diffwave	DiffTime	Cot-GAN
ETTh	Context-FID Score (Lower the Better)	64	0.631±.058	1.130±.102	<u>0.827±.146</u>	1.543±.153	1.279±.083	3.008±.277
		128	0.787±.062	1.553±.169	<u>1.062±.134</u>	2.354±.170	2.554±.318	2.639±.427
		256	0.423±.038	5.872±.208	<u>0.826±.093</u>	2.899±.289	3.524±.830	4.075±.894
	Correlational Score (Lower the Better)	64	<u>0.082±.005</u>	0.483±.019	0.067±.006	0.186±.008	0.094±.010	0.271±.007
		128	<u>0.088±.005</u>	0.188±.006	0.054±.007	0.203±.006	0.113±.012	0.176±.006
		256	<u>0.064±.007</u>	0.522±.013	0.046±.007	0.199±.003	0.135±.006	0.222±.010
	Discriminative Score (Lower the Better)	64	0.106±.048	0.227±.078	0.171±.142	0.254±.074	<u>0.150±.003</u>	0.296±.348
		128	0.144±.060	0.188±.074	<u>0.154±.087</u>	0.274±.047	0.176±.015	0.451±.080
		256	0.060±.030	0.442±.056	<u>0.178±.076</u>	0.304±.068	0.243±.005	0.461±.010
	Predictive Score (Lower the Better)	64	0.116±.000	0.132±.008	<u>0.118±.004</u>	0.133±.008	<u>0.118±.004</u>	0.135±.003
		128	0.110±.003	0.153±.014	<u>0.113±.005</u>	0.129±.003	0.120±.008	0.126±.001
		256	0.109±.013	0.220±.008	<u>0.110±.027</u>	0.132±.001	0.118±.003	0.129±.000
Energy	Context-FID Score (Lower the Better)	64	0.135±.017	1.230±.070	2.662±.087	2.697±.418	<u>0.762±.157</u>	1.824±.144
		128	0.087±.019	2.535±.372	3.125±.106	5.552±.528	<u>1.344±.131</u>	1.822±.271
		256	0.126±.024	5.032±.831	3.768±.998	5.572±.584	4.735±.729	<u>2.533±.467</u>
	Correlational Score (Lower the Better)	64	0.672±.035	3.668±.106	1.653±.208	6.847±.083	<u>1.281±.218</u>	3.319±.062
		128	0.451±.079	4.790±.116	1.820±.329	6.663±.112	<u>1.376±.201</u>	3.713±.055
		256	0.361±.092	4.487±.214	<u>1.279±.114</u>	5.690±.102	1.800±.138	3.739±.089
	Discriminative Score (Lower the Better)	64	0.078±.021	0.498±.001	0.499±.000	0.497±.004	<u>0.328±.031</u>	0.499±.001
		128	0.143±.075	0.499±.001	0.499±.000	0.499±.001	<u>0.396±.024</u>	0.499±.001
		256	0.290±.123	0.499±.000	0.499±.000	0.499±.000	<u>0.437±.095</u>	0.498±.004
	Predictive Score (Lower the Better)	64	0.249±.000	0.291±.003	0.302±.001	<u>0.252±.001</u>	<u>0.252±.000</u>	0.262±.002
		128	0.247±.001	0.303±.002	0.318±.000	0.252±.000	<u>0.251±.000</u>	0.269±.002
		256	0.245±.001	0.351±.004	0.353±.003	<u>0.251±.000</u>	<u>0.251±.000</u>	0.275±.004

Figure 5: Examples of time series imputation (1st row) and forecasting (2nd row) for ETTh and Energy datasets. Green and gray colors correspond to Diffusion-TS and CSDI, respectively.

五、收获与启发

5.1 收获

1. 结构可解释性驱动建模：Diffusion-TS 将趋势与季节性分解显式嵌入扩散去噪网络，通过趋势合成层与傅里叶合成层分别建模低频趋势与周期结构，模型能显式学习时间序列的趋势、周期项和残差等成分。在提升生成质量的同时，为条件生成提供了可控的语义结构。
2. 重构导向的训练目标：模型直接预测原始样本，并引入频域傅里叶损失，实现时域与频域的联合约束，显著增强了周期性时间序列的生成与条件保持能力。
3. 即插即用的条件生成机制：基于重构引导的采样策略在推断阶段通过梯度约束注入条件，无需重训练即可适配插补、预测等多任务场景。

5.2 启发

1. **分层条件控制**：可借鉴其分解架构，分别对趋势项与周期项施加差异化条件约束，实现更细粒度的生成控制。在语音分离问题中，对于不同声音音素特征，是否可以进行差异化条件约束。
2. **自适应损失权重**：根据条件信息完整度动态调整时域与频域损失权重，以充分利用结构先验。在语音分离问题中，对应不同的噪声或者频率特征，动态调整不同特征的损失权重。
3. **多尺度条件融合**：通过局部条件与全局条件的多尺度注入，例如，局部条件（如近期历史值）通过交叉注意力注入解码器，而全局条件（如整体统计量）作为扩散步长的嵌入向量，从而增强对长短周期条件的协同利用能力。